Trout++: Robust Asynchronous Two-Round ECDSA for Arbitrary Thresholds

Ariel Nof[1] and Luke Parker[2]

[1] Bar-Ilan University, Ramat-Gan, Israel
ariel.nof@biu.ac.il
[2] Serai DEX, Florida, USA
lukeparker@serai.exchange

Abstract. We present Trout++, a complete threshold signing suite for ECDSA signatures. Trout++ descends from the recent Trout protocol (Dahari-Garbian, Nof, and Parker, ACM CCS 2025) and inherits its transparent setup, two-round structure, and strong security guarantees, while introducing several significant improvements.

Unlike Trout, Trout++ offers pre-signing, where the first round is key-, signing-set-, and message- independent. This property is not only important in its own right but also enables us to apply the ROAST transformation (Ruffing, Ronge, Jin, Schneider-Bensch, and Schröder, ACM CCS 2022) to our protocol, yielding the *first* arbitrary-threshold (including with a dishonest majority), robust, asynchronous signing protocol for ECDSA signatures.

Second, we introduce several optimizations to the building blocks in Trout that reduce both bandwidth and computation. Our benchmark results show that our implementation of Trout++ is approximately twice as fast *and* twice as small as our prior implementation of Trout was. We also update our constant-time implementation, achieving execution of Trout++ *ten times faster* than we executed Trout in constant-time.

Finally, we show how Trout++ can be composed with an account derivation scheme that does not require additional setups per derivation, enabling more practical solutions for managing key material.

In total, we achieve complexities and functionality comparable to, and closing the gap with, leading solutions for Schnorr signatures (such as FROST).

1 Introduction

ECDSA, the Elliptic Curve Digital Signature Algorithm, is a widely-deployed signature scheme. Standardized by the National Institute of Standards and Technology, it is used for Transport Layer Security (TLS) encryption, authenticating Domain Name System (DNS) records via DNSSEC, and is frequently used for signing transactions in cryptocurrency networks such as Bitcoin and Ethereum.

In the setting of *threshold* ECDSA, the private signing key is distributed across n parties with a threshold t, such that only a subset of $t + 1$ parties can reconstruct the key. To sign messages, any $t + 1$ parties must run a secure

protocol such that only the generated signature is learned by the parties, but no information about the private key is revealed. The problem of constructing efficient threshold ECDSA protocols has attracted considerable attention in the last decade or so, the challenge stemming from the fact that, unlike schemes for Schnorr, EdDSA, and BLS, the ECDSA signature is not "MPC-friendly". Producing a signature requires computing the multiplicative inverse of a nonce and multiplying it by (a derivative of) the private key. These are non-linear operations that are rather costly when computed using generic MPC techniques.

The first solution for arbitrary thresholds was introduced in [18] and [2], but with an impractical distributed key generation protocol (requiring many parties to perform a distributed Paillier key generation). In 2018, concurrent works by Gennaro and Goldfeder [17], Lindell and Nof [22], and Doerner et al. [15] presented the first arbitrary-threshold protocols for multi-party ECDSA with practical distributed key generation and signing. Since then, a long line of works proposing new protocols has emerged, aiming to improve efficiency across various metrics, offering different assumptions and cryptographic tools, and enhancing the functionality provided.

Recently, the first two-round signing protocols for threshold ECDSA were independently published: Trout [13] and the protocol by Lyu, Li, Zhou, and Deng [23]. These were the first protocols to achieve the optimal round complexity for ECDSA signatures, and in the bulletin board model, each achieved constant per-participant upload (a single message per round of size independent to the threshold, signing set). There were some differences between the two works. Unlike Trout, [23] achieved the desired property of pre-signing, but with weaker security guarantees compared to Trout. In particular, Trout achieved identifiable abort (IA), meaning the parties can identify a cheater when the protocol does not terminate successfully. Trout also offered a publicly verifiable transcript of linear complexity and relies on simulation-based proofs (whereas the security proof in [23] relies on game-based definitions).[3]

Given the above, the gap between state-of-the-art threshold ECDSA protocols and their MPC-friendly counterparts, such as Schnorr, seems almost closed. Indeed, for Schnorr, the best existing protocols [3, 21] also require two rounds.

However, some significant gaps still remain. Despite comparable complexities, the popular and widely deployed threshold Schnorr protocol, FROST [21] (as extended via following works and ecosystem adoption), still offers functionality not generally obtained by ECDSA threshold protocols. First, FROST supports account derivation as extended in Re-Randomized Frost [19], allowing the presentation of multiple identities without running additional setups or distributed key-generation protocols. Not only does this avoid executing protocols with non-trivial computational burden, it also simplifies recovery procedures by eliminating repeated setups that each must be securely stored. Since 2012, Bitcoin

[3] Simulation based-proofs have the advantage of not requiring the forking lemma, rewinding, and so on, in order to prove security. In addition, it significantly reduces the required assumptions and provides tight security including during concurrent composition.

	Communication per party per execution		Presigning	IA	Security Type	Robust & Asynchronous
	w/o IA	IA				
Trout [13]	3.9 KB	6.3 KB	✗	✓	Simulation	✗
Lyu et al. [23]	1.9 KB	-	✓	✗	Game-based	✗
Trout++	1.8 KB	3.8 KB	✓	✓	Simulation	✓

Table 1. Comparison between Trout++ and the previous works which achieved threshold ECDSA in two rounds. Trout++ is only robust and asynchronous when composed with ROAST (requiring IA), which will perform multiple executions of Trout++.

Improvement Proposal 32 (BIP32) [4] has detailed a scheme for Bitcoin wallets to generate multiple Bitcoin addresses from a single private key, and this has become expected functionality for users.

Furthermore, FROST can be transformed into a robust, asynchronous signing protocol via ROAST [25]. ROAST is a transform which may be applied to any protocol that meets certain security definitions and structural properties. Most notably, it requires the underlying protocol to be a two-round signing protocol unforgeable during concurrent executions, with identifiable aborts, and with support for signing-set- and message- independent presigning Unfortunately, as explained above, both [13] and [12] achieve only some of these properties. We note that the first round of Trout relies crucially on a primitive called exponent VRF [3], which takes the signing-set and message as inputs, so achieving presigning would initially appear nontrivial. Similarly, the protocol of [12] relies on a primitive called non-interactive multiplication [5], which is not known to feasibly allow identifying cheaters.

1.1 Our Contributions

In this work, we close the gaps listed above by presenting Trout++, a multiparty ECDSA protocol for arbitrary thresholds, with the following features (see Table 1 for a summary and comparison):

- The signing protocol has two rounds, with the first round key-, signing-set-, and message- independent, enabling pre-signing.
- It achieves identifiable aborts (IA).
- It has constant per party communication of just 3.8 KB with IA and 1.8 KB without IA in the bulletin board model.
- It is proven secure under strong simulation-based proofs.
- It achieves robustness and works in the asynchronous setting via the ROAST transformation. The resulting composition takes $2 \cdot (n-(t+1))+3$ rounds at most with $\mathcal{O}((t+1) \cdot (n-t)^2 \cdot (n+((t+1) \cdot \kappa)))$ communication complexity[5].

In addition, we show how Trout++ can be used securely with an account-derivation scheme, allowing the creation of an arbitrary number of identities despite a constant number of key generations.

[4] https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki

[5] n = number of parties, t = corruption threshold, κ = security parameter

We have implemented Trout++ and benchmarked it to show its practicality and improvements over Trout. With an efficient, composable protocol that supports account derivation, and offers strong security guarantees, we close the gap between threshold ECDSA and Schnorr signing in a way not seen before.

Technical Contribution. As its name suggests, Trout++ builds upon the approach of Trout, while extending it to support pre-signing and reducing its bandwidth. This is achieved without giving up on the strong security guarantees of Trout. Recall that for a message m and a signing key x, the ECDSA signature is computed as $k^{-1} \cdot (\mathcal{H}(m) + r \cdot x)$, where r is the x-coordinate of $R = k \cdot E$ and E is the group generator. At a high level, the first round of Trout is dedicated to producing R, an encryption of k, and a commitment to a random mask. In the second round, the parties perform a secure protocol called *scaled decryption*, which multiplies encrypted and committed data *and* decrypts the result in a single round. The public output of this step enables the parties to locally compute the signature. We present improvements to Trout for both steps:

- *Computing $R = k \cdot E$ in one round*: We remove the non-trivial signing-set- and message- dependent exponent VRF [3] used in Trout, replacing it with the methodology of [1], which is more efficient and does not require such context. This subprotocol works by first producing R in a naive way and then re-randomizing it by calling a random oracle over the transcript twice to receive random values ρ and μ. The final nonce is set to $\rho \cdot R + \mu \cdot E$. In [1], it was shown that this only incurs a small constant loss in security compared to standard ECDSA signing. Unlike [23], who also used this methodology, we prove that the protocol realizes a carefully defined ideal signing functionality with presigning, rather than proving security via a game-based definition.
- *Improved scaled decryption:* In Trout, the scaled decryption protocol enables multiplication and decryption in a single round over two inputs, one of which is encrypted in a CLT ciphertext [10] and the other committed to with what is approximately a Pedersen commitment. We improve the above by replacing the public-key encryption with a one-time symmetric encryption scheme and a new, optimized commitment scheme. Then, we present a new scaled decryption protocol that works over these two building blocks. Note that this new protocol not only reduces bandwidth and computation but also avoids the use of public-key encryption in our protocol, thereby removing the need to generate a CLT public key and yielding a simpler and more efficient setup. The usage of symmetric encryption in place of public-key encryption may be of inspiration to other MPC protocols.

2 Preliminaries

Notations. We use lower-case letters to represent scalars, and upper-case letters to denote group elements. Variables used as randomness within ciphertexts/commitments are denoted with greek letters. We use $||$ to denote string concatenation. For a prime q, let $\mathbb{F}_q$ denote a finite field of size q. We use κ to

denote a fixed security parameter. We use $[n]$ to denote the set $\{1, ..., n\}$. Composing group elements is notated additively with scaling notated multiplicatively.

Security and network model. For Trout++'s functionality, within our security proofs, we initially assume a synchronous network with authenticated point-to-point channels between the parties. In the version of the protocol that achieves IA, we instead assume a synchronous broadcast channel, which enables the parties to publish their messages (as required to agree on the transcript).

We use the standard real/ideal world paradigm to prove the security of our protocols [7]. In particular, for each of our protocols, we show that it realizes with malicious security an ideal functionality that we define later. When we prove a protocol realizes the ideal functionality with *selective (non-unanimous)* abort, it means that the ideal-world adversary can prevent each of the honest parties from obtaining their output. In contrast, when we prove security with *identifiable* abort, then either all the honest parties receive their output or all the honest parties abort while receiving an index of a corrupted party to blame.

When our protocol (with identifiable aborts) is composed with the ROAST transform [25], the resulting protocol works over an asynchronous network as defined by ROAST. Our security proofs continue to compose even with this change in network model.

2.1 Groups and Assumptions

Elliptic Curves Let $\mathsf{Gen}_{\mathsf{EC}}$ be an algorithm that, given an input 1^κ, outputs the tuple

$$\mathsf{ppEC} = (\hat{E}, q, E)$$

where $\hat{E}$ is a cyclic group with prime order q and E is a generator of $\hat{E}$. In this paper, the group operation is notated additively, and each point A in $\hat{E}$ is of the form $A = (a, \cdot)$ for some $a \in \mathbb{N}$.

Class Groups Let $\mathsf{Gen}_{\mathsf{CL}}$ be an algorithm that takes 1^κ and an odd prime order q (with length a function of κ) as an input, and outputs the tuple

$$\mathsf{ppCL} = (\bar{s}, \hat{G}, \hat{G}_q, \hat{H}, G, G_q, H).$$

The set $\hat{G}$ with the addition operation is a finite abelian group of order $q \cdot s'$, where the length of s' is a function of κ and $gcd(q,\ s') = 1$. The value $\bar{s}$ is the upper bound of s'. One can decide if an element is in $\hat{G}$ in polynomial time. The set $\hat{H}$ with the addition operation is the unique cyclic subgroup of $\hat{G}$ of order q, generated by H. The group $\hat{G}_q := \{q \cdot x,\ x \in \hat{G}\}$ is the subgroup of order s' of $\hat{G}$, generated by G_q. It thus holds that $\hat{G} = \hat{G}_q \times \hat{H}$ and $G = G_q + H$ is a generator of $\hat{G}$. The discrete logarithm problem in $\hat{H}$ can be solved by a polynomial time algorithm $\mathsf{DLsolve}$, i.e., for each $x \in \mathbb{F}_q$ it holds that $x = \mathsf{DLsolve}(x \cdot H)$.

Distributions. We define two distributions $\mathcal{D}_q$ and $\mathcal{D}$. Let $\mathcal{D}_q$ (resp., $\mathcal{D}$) denote the distribution which satisfies that $\{r \cdot G_q \ : \ r \leftarrow D_q\}$ (resp., $\{r \cdot G \ : \ r \leftarrow D\}$) is close to uniformly sampling random elements in $\hat{G}_q$ (resp., $\hat{G}$). For setting concrete parameters for these distributions, see [10].

Assumptions. The unknown order assumption says that it is hard to find the order of an element in the subgroup with an unknown order.

Definition 1 (Unknown Order Assumption). *Let $\mathcal{A}$ be a PPT algorithm and consider the following experiment: The challenger runs $\mathsf{Gen}_{\mathsf{CL}}(1^\kappa)$ to obtain ppCL, $\mathcal{A}$ is given $(\mathsf{ppCL}, \mathsf{DLsolve})$ and outputs Z and e. We say that $\mathcal{A}$ wins if $Z \in \hat{G}_q$, Z is not the identity, $e \neq 0$, and $e \cdot Z$ is the identity. Denote by $\mathsf{Adv}^{ORD}_{\mathcal{A}}(\kappa)$ the success probability of $\mathcal{A}$. We say that the unknown order assumption is hard in $\hat{G}$ if for all PPT adversaries $\mathsf{Adv}^{ORD}_{\mathcal{A}}(\kappa)$ is negligible in κ.*

The HSM assumption says that a PPT adversary cannot distinguish between random elements in $\hat{G}$ and $\hat{G}_q$.

Definition 2 (Hard Subgroup Membership Assumption (HSM)). *Let $\mathcal{A}$ be a PPT algorithm and consider the following experiment: The challenger runs $\mathsf{Gen}_{\mathsf{CL}}(1^\kappa)$ to obtain ppCL, samples x and x' from $\mathcal{D}_q$ and $\mathcal{D}$ respectively and $b \in \{0,1\}$. Then, it sets $Z_0 = x \cdot G_q$ and $Z_1 = x' \cdot G$. $\mathcal{A}$ is given $(\mathsf{ppCL}, Z_b, \mathsf{DLsolve})$ and outputs a bit b'. We say that $\mathcal{A}$ wins if $b' = b$. Denote by $\mathsf{Adv}^{HSM}_{\mathcal{A}}(\kappa)$ the success probability advantage of $\mathcal{A}$ over an adversary who simply guesses b. We say that the HSM assumption is hard in $\hat{G}$ if for all PPT adversaries $\mathsf{Adv}^{HSM}_{\mathcal{A}}(\kappa)$ is negligible in κ.*

2.2 The ECDSA signature

The ECDSA signature scheme is computed over the elliptic curve $\hat{E}$ of prime order q. Let $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_q$. Then, the scheme consists of the following three algorithms:

$\underline{\mathsf{KeyGen}(1^\kappa)}$: Choose $x \in \mathbb{F}_q$. The output is a pair (x, X), where $X = x \cdot E$. The signing key is x while X is the verifying key.

$\underline{\mathsf{Sign}(x; m)}$: Given a message m, choose a random k and set $R = k \cdot E$. Let r be the x-coordinate of $R \mod q$. Output (r, s), where $s = k^{-1} \cdot (\mathcal{H}(m) + r \cdot x)$

$\underline{\mathsf{Verify}(X; m, \sigma)}$: Given a message m and a signature $\sigma = (r, s)$, check $0 < r, s < q$, set $z = ((\mathcal{H}(m) \cdot s^{-1}) \cdot E) + ((r \cdot s^{-1}) \cdot X)$ and check that z is not the point at infinity and its x-coordinate modulo q is equal to r. If it holds, output 1. Otherwise, output 0.

2.3 Shamir Secret Sharing and Feldman's VSS

The Shamir Secret sharing [26] with a threshold t consists of two algorithms: Share and $\mathsf{Reconstruct}$. In the Share algorithm, a dealer wishes to share a secret

$x_0 \in \mathbb{F}_q$ to parties $P_1, ..., P_N$. In the algorithm, the dealer chooses a random polynomial $f(x) = a_0 + \sum_{j=1}^{t} a_j \cdot x^j$ of degree t, under the constraint that $f(0) = a_0 = x_0$. Then, it hands each party P_i the point $f(i)$. In the $\mathsf{Reconstruct}$ algorithm, the parties send their shares to each other and use them to compute x_0 via Lagrange interpolation. We denote by $\mathcal{L}_i^S(x)$ the Lagrange coefficient that corresponds to a subset of parties S of size $t+1$, a party $i \in S$, and an evaluation point x.

Feldman's VSS [16] enables the parties to detect cheating in the sharing process. To this end, in the sharing procedure, the dealer additionally publishes the polynomial coefficients in the exponent. That is, the dealer sends each party P_i its share $f(i)$ and also publishes $A_j = a_j \cdot E$ for $j = 0, ..., t$. Hence, each party can check that its share s_i satisfies the equation $s_i \cdot E = \sum_{j=0}^{t} i^j \cdot A_j$. Formally, we define two algorithms:

- $\underline{\left((s_1, ..., s_N), \{a_j\}_{j=0}^t, \{A_j\}_{j=0}^t\right) \leftarrow \mathsf{Vss.Share}(x_0)}$: In this algorithm, the dealer runs $\mathsf{Share}(x_0)$ to get the polynomial coefficients $a_0, ..., a_t \in \mathbb{F}_q$ (where $a_0 = x_0$), and then computes $A_j = a_j \cdot E$ for each $j \in \{0, ..., t\}$.
- $\underline{\beta \leftarrow \mathsf{Vss.Vrfy}(s_i, \{A_j\}_{j=0}^t)}$: In this algorithm, party P_i checks that $s_i \cdot E = \sum_{j=0}^{t} i^j \cdot A_j$. If the equation holds, it outputs 1. Otherwise, the output is 0.

3 Class Group Building Blocks

3.1 Class Group Encryption

We leverage a derivative of CLT encryption scheme over class groups [10]. Recall that we work over a group $\hat{G} = \hat{G}_q \times \hat{H}$, with a generator G_q for $\hat{G}_q$ and a generator H for $\hat{H}$. The order of $\hat{G}_q$ is unknown while the order q of $\hat{H}$ is known. The discrete log problem is hard in $\hat{G}_q$ but easy in $\hat{H}$. Typically, for the secret key y^{cl} and public key $Y^{\mathsf{cl}} = y^{\mathsf{cl}} \cdot G_q$, CLT ciphertexts consist of two group elements $(C_1,\ C_2) = (\alpha \cdot G_q,\ \alpha \cdot Y^{\mathsf{cl}} + m \cdot H)$. It is possible to decrypt the ciphertext by taking $C_2 - y^{\mathsf{cl}} \cdot C_1$ and invoking the $\mathsf{DLsolve}$ algorithm over the result. In our protocol, the parties do not use y^{cl} at all and perform decryption using the randomness α. That is, decryption is carried out by taking $C_2 - \alpha \cdot Y^{\mathsf{cl}}$ and, as before, invoking the $\mathsf{DLsolve}$ over the result. Hence, we do not need to compute nor store the first element at all! This allows us to reduce both bandwidth usage and computational cost. We note this transforms CLT's public-key encryption scheme into a symmetric encryption scheme, where the randomness is needed for both encryption and decryption. As Trout++ is not a robust signing protocol itself, we already require a consistent signing set across rounds, meaning the participants performing encryption in round one are also the set performing decryption in round two (and therefore already know the randomness). This removes the need for a public-key encryption scheme, solely an additively-homomorphic symmetric encryption scheme.

Encrypt: Given a message m, choose a random $\alpha \in \mathcal{D}_q$ and output

$$\mathsf{Enc}(\alpha, m) = \alpha \cdot G_q + m \cdot H$$

Decrypt: Given a ciphertext C and a random one-time key α, output

$$\mathsf{Dec}_\alpha(C) = \mathsf{DLsolve}(C - (\alpha \cdot \hat{G}_q))$$

Note that this encryption scheme is *computationally* hiding under the HSM assumption. To see this observe that an encryption of 0 is an element in $\hat{G}_q$, whereas an encryption of any other message is an element in $\hat{G}$. Hence, encryptions of two such messages are indistinguishable if it is not possible to distinguish an element of $\hat{G}$ as an element of the subgroup $\hat{G}_q$.

As for binding, given a ciphertext C, if it is possible to open it into two pairs (α, m) and (α', m'), we have $\alpha \cdot G_q + m \cdot H = \alpha' \cdot G_q + m' \cdot H$. This implies $\alpha \cdot G_q = \alpha' \cdot G_q + (m - m') \cdot H$. However, now we have on the left side an element in $\hat{G}_q$ without any term in $\hat{H}$. Hence, the equation can hold if and only if $m - m' \equiv 0 \bmod q$. The scheme is thus perfectly binding.

Lemma 1. *The* Enc *scheme is perfectly binding. Under the HSM assumption, it is computationally hiding.*

3.2 Class Group Commitment

We now define a statistically hiding, computationally binding commitment scheme for values $\in \mathbb{F}_q$, under the assumption that G_q is a generator of $\hat{G}_q$ and the *order of* $\hat{G}_q$ *is unknown*. To commit to a value $m \in \mathbb{F}_q$, sample $\alpha \leftarrow \mathcal{D}_q$ and compute

$$\mathsf{OptCom}(\alpha, m) = \alpha \cdot (q \cdot G_q) + m \cdot G_q.$$

As the order of $\hat{G}_q$ is coprime to q except with negligible probability [11, Conjecture 5.10.1], $q \cdot G_q$ is statistically also a generator of $\hat{G}_q$. Since α is chosen from $\mathcal{D}_q$, the commitment is statistically close to uniform, hence the commitment is statistically hiding.

To show binding, assume that the committer can open the same commitment with two openings $\alpha_1,\ m_1$ and $\alpha_2,\ m_2$ such that $m_1 \neq m_2,\ \mathsf{OptCom}(\alpha_1, m_q) = \mathsf{OptCom}(\alpha_2, m_2)$. This means that $(\alpha_1 - \alpha_2) \cdot (q \cdot G_q) + (m_1 - m_2) \cdot G_q$ is the identity element and therefore $(\alpha_1 - \alpha_2) \cdot q + (m_1 - m_2)$ is the order of the group, thus breaking the unknown order assumption.

Lemma 2. OptCom *is statistically hiding. Under the unknown order assumption, it is computationally binding.*

Comparison to Trout. In Trout [13], the commitment used in the protocol was of the form $\alpha \cdot G_q + m \cdot Y$, where Y is the public key of the encryption scheme (thus its discrete logarithm with respect to the generator G_q is secret). This implies scaling two group elements, whereas with our optimized commitment, only a single scaling is required since the committer can compute $\alpha \cdot q + m$ and then perform a single scaling of G_q. Note however that the commitment in Trout was carefully designed to make the scaled decryption protocol used in the signing phase work in one round. Fortunately, we are able to construct a new scaled decryption protocol that works with the optimized commitment and symmetric encryption schemes. This will be presented in Section 6.1.

4 Zero-Knowledge Ideal Functionalities and Relations

Definition 3 (NIZKAOK protocol). *Let $\mathcal{L}$ be a NP Language and let $\mathcal{R}_{\mathcal{L}}$ be the associated relation. Let $\mathcal{H}$ be a random oracle. A non-interactive zero-knowledge argument of knowledge protocol consists of the following algorithms:*

- $\mathsf{NIZKAOK.prove}^{R}_{\mathcal{H}}(x, w) \to \pi$*: upon receiving a statement x and a witness w such that $(x, w) \in R$, the algorithm outputs a proof π.*
- $\mathsf{NIZKAOK.vrfy}^{R}_{\mathcal{H}}(x, \pi) \to \{0, 1\}$*: upon receiving a statement x and a proof π, the algorithm outputs* 1 *(accept) or* 0 *(reject).*

We note that the public statement x also includes any public parameters *required for defining the relation.*

The algorithms should satisfy the following requirements:

- **Completeness:** *If $(x, w) \in R$, and the algorithms are executed honestly, then the output of $\mathsf{NIZKAOK.vrfy}(x, \pi)$ is 1 with probability 1.*
- **Zero-knowledge:** *There exists a PPT simulator $\mathcal{S}$ such that for all PPT algorithms D and for all $(x, w) \in R$ there exists a negligible function* neg *such that:*

$$\begin{aligned}&\mathsf{Pr}\left[D^{\mathcal{H}}(x, \pi) = 1 \;\middle|\; \pi \leftarrow \mathcal{S}(x)\right] \\ &\quad - \mathsf{Pr}\left[D^{\mathcal{H}}(x, \pi) = 1 \;\middle|\; \pi \leftarrow \mathsf{NIZKAOK.prove}^{R}_{\mathcal{H}}(x, w)\right] \leq \mathsf{neg}(\kappa)\end{aligned}$$

- **knowledge extractabilty:** *There exists an extractor* Ext *such that for all PPT $\mathcal{A}$, there exists a negligible function* neg *such that:*

$$\Pr\begin{bmatrix}\mathsf{NIZKAOK.vrfy}^{R}_{\mathcal{H}}(x, \pi^*) = 1 \\ \wedge (x, w^*) \notin R_{\mathcal{L}}\end{bmatrix} \;\middle|\; \begin{matrix}(x, \pi^*) \leftarrow \mathcal{A}^{\mathcal{H}}(\mathrm{pp}) \\ w^* \leftarrow \mathsf{Ext}^{\mathcal{A}}(x, \pi^*)\end{matrix} \leq \mathsf{negl}(\kappa).$$

In this work, we use the sigma protocols of [12] (which is an improvement of [27]) with the Fiat-Shamir transform to make the protocols non-interactive. These proofs have been shown to be secure under the Adaptive Root Subgroup Assumption and Hard Subgroup Membership Assumption (see Appendix [12] for more details). ZK proofs over class groups can be complicated, since extraction traditionally requires computing an inverse in a group of unknown order. Earlier works [9] used a single-bit challenge, which required many repetitions of the proof to obtain a sufficiently small soundness error, thereby making ZK proofs for class groups slow and expensive. The works of [27, 12] were able to overcome this problem by responding over prime fields determined by the challenge, significantly improving both the proof size and the computation time while maintaining the ability to fully extract the witness. This opened the door to practical and concretely-efficient threshold signing protocols using class groups, without sacrificing notions of security, as in this work.

Relations We list several instantiations which we use in the protocol.

– Knowledge of discrete logarithm over an elliptic curve

$$R_{\mathsf{Dlog}} = \{(A_0, ..., A_t, (a_0, ..., a_t)) : \ \forall j \in \{0, ..., t\} : \ A_j = a_j \cdot E\}$$

– Knowledge of committed value

$$R_{\mathsf{ComKwlg}} = \{(B, (\beta, b)) : \ B = \mathsf{OptCom}(\beta, b)\}$$

– Equality between a CL encrypted value and EC discrete log

$$R_{\mathsf{CL-EC}} = \{(A, C, (\alpha, a)) : \ A = a \cdot E \ \wedge \ C = \mathsf{Enc}(\alpha, a)\}$$

– Computing affine operations over ciphertexts using committed values

$$R_{\mathsf{affCom}} = \{(F, D, C, B, A, (\alpha, \beta, b)) : \\ F = (\beta \cdot q + b) \cdot A - \alpha \cdot B \ \wedge \ C = \mathsf{Enc}(\alpha, -) \ \ \wedge \ D = \mathsf{Com}(\beta, b)\}$$

4.1 String Commitment

In our key generation protocol, we require the parties to commit to their ZK proofs before revealing it to the other parties. Hence, we assume the parties have an access to an ideal commitment functionality $\mathcal{F}_{\mathsf{com}}$, as formally defined in Functionality 1. Any UC-secure commitment can realize $\mathcal{F}_{\mathsf{com}}$. In particular, it can be trivially realized using a random oracle $\mathcal{H}$, by defining $\mathsf{Com}(x) = \mathcal{H}(\tau, x)$ where $\tau \leftarrow \{0,1\}^{\kappa}$ is random.

Functionality 1 ($\mathcal{F}_{\mathsf{com}}$ - The Commitment Functionality)

The functionality works with parties $P_1, ..., P_n$.

– Upon receiving $(\mathsf{sid}, \mathsf{commit}, x, i)$ from P_i: if sid has not been used before by P_i, store (sid, i, x) and send $(\mathsf{sid}, \mathsf{receipt}, i)$ to the other parties.
– Upon receiving $(\mathsf{sid}, \mathsf{decommit}, i)$ from P_i retrieve (sid, i, x) and send $(\mathsf{sid}, \mathsf{decommit}, i, x)$ to all parties.

5 Key Generation and Setup Protocols

In this section, we present our key generation and setup protocols. The key generation protocol is a traditional protocol to produce an ECDSA signing key in an unbiased way, resulting in each party holding a share of the key. The setup protocol produces an encryption of each party's share for use in the signing protocol. We explicitly delineate these two protocols as for key generation, any unbiased key generation protocol that outputs the public key and the shares of each party would suffice, allowing the user to choose which to compose into the resulting Trout++ Distributed Key Generation (DKG) protocol. Protocol 2 is our setup protocol that inputs the public ECDSA key and its shares, outputting the ciphertexts for each share.

Remark 1. We note the setup protocol can be executed in parallel to the first round of the signing protocol, as in our signing protocol, the first round is key-independent. This allows executing Trout++ without prior additional setup.

Protocol 2 (Trout++ Setup)

- **Public input:** $\mathsf{sid}, \mathsf{ppEC}, \mathsf{ppCL}, X, \{X_i\}_{i=1}^N$
- **Private input:** x_i
- **The protocol:** Each party P_i works as follows:
  - **Round 1:**
    1. Choose a random $\delta_i \in \mathcal{D}_q$ and compute $C_i = \mathsf{Enc}(\delta_i, x_i)$.
    2. Compute $\pi_i \leftarrow \mathsf{NIZKAOK.prove}_{\mathcal{H}}^{R_{\mathsf{CL-EC}}}(X_i, C_i, (\delta_i, x_i))$.
    3. Send $(\mathsf{sid}, C_i, \pi_i)$ to all parties.
  - **Computing the output:** Upon receiving $(\mathsf{sid}, C_k, \pi_k)$ for all $k \in [N]$:
    1. For each $k \in [N]$: compute $\beta_k \leftarrow \mathsf{NIZKAOK.vrfy}_{\mathcal{H}}^{R_{\mathsf{CL-EC}}}(X_k, C_k, \pi_k)$. If $\beta_k = 0$, send abort to all parties and halt.
    2. Output $X, \{X_k\}_{k=1}^N, \mathsf{ppCL}, \{C_k\}_{k=1}^N, \delta_i, x_i$.

Next, in Protocol 3, we describe the traditional DKG which outputs Shamir secret shares of the signing key, their corresponding commitments, and the verification key, needed for the setup protocol.

Protocol 3 (Unbiased Key Generation)

- **Public input:** $\mathsf{sid}, \mathsf{ppEC}$
- **The protocol:** Each party P_i works as follows:
  - **Round 1:**
    1. Choose a random x_i and run $\mathsf{Vss.Share}(x_i)$ to receive $s_{i,1}, ..., s_{i,N}, \{a_{i,j}\}_{j=0}^t, \{A_{i,j}\}_{j=0}^t$ where $a_{i,0} = x_i$.
    2. Compute $\pi_i \leftarrow \mathsf{NIZKAOK.prove}_{\mathcal{H}}^{R_{\mathsf{Dlog}}}\left(\{A_{i,j}\}_{j=0}^t, \{a_{i,j}\}_{j=0}^t\right)$
    3. Send $\left(\mathsf{sid}, \mathsf{commit}, \{A_{i,j}\}_{j=0}^t, \pi_i, i\right)$ to $\mathcal{F}_{\mathsf{com}}$.
  - **Round 2:** Upon receiving $(\mathsf{sid}, \mathsf{receipt}, k)$ for each $k \in [N]$:
    1. Send $(\mathsf{sid}, \mathsf{decommit}, i)$ to $\mathcal{F}_{\mathsf{com}}$.
    2. Send $s_{i,k}$ to party P_k.
  - **Computing the output:** Upon receiving $(\mathsf{sid}, \mathsf{decommit}, \{A_{k,j}\}_{j=0}^t, \pi_k, k)$ from $\mathcal{F}_{\mathsf{com}}$ for each $k \in [N]$ and $s_{k,i}$ from each P_k:
    1. For each $k \in [N]$: compute $\beta_k \leftarrow \mathsf{NIZKAOK.vrfy}_{\mathcal{H}}^{R_{\mathsf{Dlog}}}(\{A_{k,j}\}_{j=0}^t, \pi_k)$. If $\beta_k = 0$, send abort to the other parties and halt.
    2. For each $k \in [N]$: run $\beta_k \leftarrow \mathsf{Vss.Vrfy}\left(s_{k,i}, \{A_{k,j}\}_{j=0}^t\right)$. If $\beta_k = 0$, send abort to the other parties and halt.
    3. Output $X = \sum_{k=1}^N A_{k,0}$, $x_i = \sum_{k=1}^N s_{k,i}$ and $X_i = \sum_{k=1}^N \left(\sum_{j=0}^t k^j \cdot A_{k,j}\right)$.

The above is straightforward and traditional, occurring in two rounds. However, there are other alternatives that can be used instead of the above protocol. As Trout++ already assumes an additively-homomorphic encryption scheme, one could employ the publicly-verifiable scheme detailed in [8]. Alternatively, an exponent-Verifiable Random Function (eVRF, [3]) may be used to produce an unbiased key in one round, and the same techniques can be used to construct a publicly-verifiable encryption scheme for DKG protocols ([24], [6]).

6 Signing Protocol

6.1 Optimized Scaled Decryption

We now present our improved version of Trout's Scaled Decryption protocol, achieving the same desired properties, but with our new encryption and commitment schemes (as presented in Section 3). The lines in blue are executed only when the protocol aims to acheive identifiable abort (IA); see Section 6.4 for more information.

Protocol 4 (Optimized Scaled Decryption)

The protocol is executed by a set of parties S indexed by $\{\ell_1, ..., \ell_n\}$ (s.t. $n = t{+}1 \leq N$). In the protocol, the notation P_i is used to denote the party P_{ℓ_i}.

- **Public input:** $\mathsf{ppCL}, \{B_j\}_{j=1}^n, \{A_j\}_{j=1}^n$
- **Private input of each P_i:** $\alpha_i, \beta_i \in \mathcal{D}_q$ and $a_i, b_i \in \mathbb{F}_q$ such that

$$A_i = \alpha_i \cdot G_q + a_i \cdot H, \qquad B_i = \mathsf{OptCom}(\beta_i,\ b_i)$$

- **The protocol:** Let

$$A = \sum_{j=1}^{n} A_j = \alpha \cdot G_q + a \cdot H, \qquad B = \sum_{j=1}^{n} B_j = \mathsf{OptCom}(\beta, b)$$

Each party P_i does the following:

1. Compute:
   - $F_i = ((\beta_i \cdot q) + b_i) \cdot A - \alpha_i \cdot B$
   - IA-mode: $\pi_i \leftarrow \mathsf{NIZKAOK.prove}_{\mathcal{H}}^{R_{\mathsf{affCom}}}(F_i, B_i, A_i, B, A, (\alpha_i, \beta_i, b_i))$.
2. Send (sid, F_i) to all parties.
   IA-mode: send $(\mathsf{sid}, F_i, \pi_i)$ to all parties.
3. Upon receiving $(\mathsf{sid}, F_j, \pi_j)$ for all $j \in [n]$.
   (a) IA-mode: For each $j \in [n]$: compute $\beta_j \leftarrow \mathsf{NIZKAOK.vrfy}_{\mathcal{H}}^{R_{\mathsf{affCom}}}(F_j, B_j, A_j, B, A, \pi_j)$. If $\beta_j = 0$, send abort to all parties and halt.
   (b) Compute $F = \sum_{j=1}^{n} F_j$.
   (c) $c = \mathsf{DLsolve}(F)$

- **Output:** c

Claim. If all parties act honestly, then $c = a \cdot b$.

Proof. To prove the claim, we show that $F = (a \cdot b) \cdot H$. This holds since

$$F = \sum_{i=1}^{n} F_i = \sum_{i=1}^{n}(((\beta_i \cdot q) + b_i) \cdot A - \alpha_i \cdot B) =$$

$$= \sum_{i=1}^{n}(((\beta_i \cdot q) + b_i) \cdot \alpha \cdot G_q + ((\beta_i \cdot q) + b_i) \cdot a \cdot H - \alpha_i \cdot (\beta \cdot q \cdot G_q + b \cdot G_q))$$

$$= \sum_{i=1}^{n}(((\beta_i \cdot q) + b_i) \cdot \alpha \cdot G_q + b_i \cdot a \cdot H - \alpha_i \cdot ((\beta \cdot q + b) \cdot G_q)) = a \cdot b \cdot H$$

6.2 The Signing Protocol

In this section, we present our signing protocol. Recall that the signature is $s = k^{-1} \cdot (\mathcal{H}(m) + r \cdot x)$ where r is the x-coordinate of $k \cdot E$ reduced into the scalar field. As in previous works, we leverage Beaver's trick to compute the signature. Specifically, the parties compute $u \cdot k$ and $u \cdot (\mathcal{H}(m) + r \cdot x)$ in parallel, reveal the results, and then each party can locally compute $s = (u \cdot k)^{-1} \cdot (u \cdot (\mathcal{H}(m) + r \cdot x) = k^{-1} \cdot (\mathcal{H}(m) + r \cdot x)$.

We note that as in [13], the parties do not compute $u \cdot (\mathcal{H}(m) + r \cdot x)$, but $u \cdot (\mathcal{H}(m) \cdot r^{-1} + x)$, and only when the final signature is revealed, the parties locally multiply it with r to correct the result. Since the parties hold an encryption of x (as it must be kept secret), this optimization trades an expensive scaling of a ciphertext with a relatively cheap finite field inversion.

To perform the above in two rounds, the parties generate the nonce within the first round. In addition, each party commits to its share of u and encrypts its share of k in the first round. Thus, at the beginning of the second round, the parties can decide the nonce and compute $\mathsf{Enc}(k)$, $\mathsf{OptCom}(u)$, and $\mathsf{Enc}(\mathcal{H}(m) \cdot r^{-1} + x)$. It thus remains to call the scaled decryption protocol (Protocol 4) twice, on $\mathsf{Enc}(k)$, $\mathsf{OptCom}(u)$ and on $\mathsf{Enc}(\mathcal{H}(m) \cdot r^{-1} + x)$, $\mathsf{OptCom}(u)$. The two executions end with the parties holding $u \cdot k$ and $u \cdot (\mathcal{H}(m) \cdot r^{-1} + x)$, which suffices for computing the signature as explained above. Thus, the protocol has two overall rounds. The formal description appears in Protocol 5.

We now provide more details on the first round, where the parties produce $R = k \cdot E$. In Trout [13], this is carried out with a cryptographic primitive called an "exponent VRF" [3] (Verifiable Random Function). Each party commits to the VRF key in the setup before computing $R_i = k_i \cdot E$ in the signing protocol, where k_i is the output of the VRF, while proving the computation was correct with respect to the commitment from the setup. Roughly speaking, since the online protocol is in fact deterministic, it is simulatable. However, the deterministic nature of the process also means that the signing set and message must be provided as input to the VRF computation, thereby preventing the desired property of a pre-signing round.

To achieve pre-signing, we replace the eVRF with a different protocol. Our starting point is the standard naive protocol as in Lyu et al. [23]. Specifically, each party simply chooses its random share of the nonce k_i and sends $R_i = k_i \cdot E$ to the other parties. However, taking $R = \sum_{i=1}^{n} R_i$ means that the adversary can bias the key, since it sees the honest parties messages before choosing its messages. To randomize the desired nonce, in a way that is not predictable by the adversary, Lyu et al. [23] suggested to call a random oracle twice. The first call takes the entire transcript as an input, and outputs ρ. The second call receives ρ as an input to output μ. Then, the final nonce is set to be $R = \rho \cdot \sum_{i=1}^{n} R_i + \mu \cdot E$.

Unlike [23], we aim to provide a simulation-based proof. However, standard simulation typically means that the nonce is randomly chosen by the trusted party and given to the parties. This cannot work for the above technique, since the adversary can still bias the nonce. As a simple example, consider an adversary who makes queries to the random oracle with its first-round messages until the

MSB of the nonce is 0 (recall that the adversary is rushing and so it receives the honest parties' messages first). Such an attack will succeed with high probability given that the adversary is allowed to make a polynomial number of queries.

To prove security of the protocol, we define an ideal functionality that enables pre-signing and randomizing of the nonce. That is, the trusted party allows the simulator to request a nonce, tweak its value and, only later, to compute the signature for a nonce that was produced earlier. Looking ahead, the ideal functionality we define in Section 6.3 has a presign and randomize command. When randomizing, the ideal-world adversary is allowed to choose the multiplicative randomization, while the trusted party chooses the additive tweak. In the proof, we show that this enables the simulator to proceed with the following strategy: every time the adversary queries $\mathcal{H}_\rho$, the simulator learns a set of possible messages from the corrupted parties. The simulator then requests a new nonce from the ideal functionality. Since it now knows all messages associated with this nonce, it can program the oracle $\mathcal{H}_\mu$'s responses to return an additive randomization that is aligned with the k_i values chosen by the corrupted parties and the μ value received from the ideal functionality.

Finally, we note that Adjedj et al. [1] showed that such a randomization of a nonce which was chosen in a presigning round incurs only a constant security loss of 2^5, compared to standard ECDSA signing, in the generic group model.

Protocol 5 (Distributed Signing)

Let $\mathcal{H}, \mathcal{H}_\mu, \mathcal{H}_\rho : \{0,1\}^* \to \mathbb{F}_q$.
The protocol is executed by a set of parties S indexed by $\{\ell_1, ..., \ell_n\}$ (s.t. $n = t+1 \leq N$). In the protocol, the notation P_i is used to denote the party P_{ℓ_i}.

- **Public input:** $\mathsf{sid}, \mathsf{ppEC}, \mathsf{ppCL}, X, \{X_j\}_{j=1}^n, \{C_j\}_{j=1}^n$.
- **Private input of each P_i:** x_i, δ_i such that $X_i = x_i \cdot E$ and $C_i = \mathsf{Enc}(\delta_i, x_i)$.
- **The protocol:**
  - **Round 1 (Presigning):**
    1. Choose random $k'_i, u_i \in \mathbb{F}_q$ and $\alpha_i, \beta_i \in \mathcal{D}_q$ and compute:
       - $R'_i = k'_i \cdot E$.
       - $K'_i = \mathsf{Enc}(\alpha_i, k'_i) = \alpha_i \cdot G_q + k'_i \cdot H$.
       - $U_i = \mathsf{OptCom}(\beta_i, u_i) = \beta_i \cdot q \cdot G_q + u_i \cdot G_q$.
    2. Compute:
       (a) $\pi_i^{R_{\mathsf{CL-EC}}} \leftarrow \mathsf{NIZKAOK.prove}_{\mathcal{H}}^{R_{\mathsf{CL-EC}}}(R'_i, K'_i, (\alpha_i, k'_i))$.
       (b) $\pi_i^{R_{\mathsf{ComKwlg}}} \leftarrow \mathsf{NIZKAOK.prove}_{\mathcal{H}}^{R_{\mathsf{ComKwlg}}}(U_i, (\beta_i, u_i))$.
    3. Send $\mathsf{msg}_i = (\mathsf{sid}, R'_i, K'_i, U_i, \pi_i^{R_{\mathsf{CL-EC}}}, \pi_i^{R_{\mathsf{ComKwlg}}})$ to all parties.
  - **Round 2 (Signing):**
    1. Upon receiving:
       - msg_j for all $j \in [n]$
       - a message m to sign

       compute:
       - For each $j \in [n]$: $\beta_j \leftarrow \mathsf{NIZKAOK.vrfy}_{\mathcal{H}}^{R_{\mathsf{CL-EC}}}(R'_j, K'_j, \pi_j^{R_{\mathsf{CL-EC}}})$. If $\beta_j = 0$, send abort to all parties and halt.
       - For each $j \in [n]$: $\beta_j \leftarrow \mathsf{NIZKAOK.vrfy}_{\mathcal{H}}^{R_{\mathsf{ComKwlg}}}(U_j, \pi_j^{R_{\mathsf{ComKwlg}}})$. If $\beta_j = 0$, send abort to all parties and halt.

* $R' = \sum_{j=1}^{n} R'_j$.
* $\rho = \mathcal{H}_\rho(m, X, \{\mathsf{msg}_j\}_{j=1}^n)$.
* $\mu = \mathcal{H}_\mu(\mathsf{sid}, \rho)$.
* $R = \rho \cdot R' + \mu \cdot E$. Let r be the x-coordinate of R reduced into $\mathbb{F}_q$.
* $k_j = k'_j$ for all $j \neq 1$ and $k_1 = k'_1 + (\rho^{-1} \cdot \mu)$.
* $K_j = K'_j$ for all $j \neq 1$ and $K_1 = K'_1 + (\rho^{-1} \cdot \mu) \cdot H$.
* $x'_j = \mathcal{L}^S_{\ell_i}(0) \cdot x_j$ for all $j \neq 1$ and $x'_1 = \mathcal{L}^S_{\ell_i}(0) \cdot x'_1 + (r^{-1} \cdot \mathcal{H}(m))$.
* $Z_j = \mathcal{L}^S_{\ell_i}(0) \cdot C_j$ for all $j \neq 1$ and $Z_1 = \mathcal{L}^S_{\ell_i}(0) \cdot C_1 + (r^{-1} \cdot \mathcal{H}(m)) \cdot H$.

2. Run Protocol 4 on public input $(\mathsf{sid}, \mathsf{ppCL}, \{U_j\}_{j=1}^n, \{Z_j\}_{j=1}^n)$ and private input $(\mathcal{L}^S_i(0) \cdot \delta_i, \beta_i, x'_i, b_i)$ to receive $u \cdot ((r^{-1} \cdot \mathcal{H}(m)) + x)$.
3. Run Protocol 4 on $(\mathsf{sid}, \mathsf{ppCL}, \{U_j\}_{j=1}^n, \{K_j\}_{j=1}^n)$ and private input $(\alpha_i, \beta_i, k_i, b_i)$ to receive $u \cdot \rho^{-1} \cdot k$.
4. Compute $s = ((u \cdot \rho^{-1} \cdot k) \cdot \rho)^{-1} \cdot (u \cdot ((r^{-1} \cdot \mathcal{H}(m)) \cdot x)) \cdot r$.
5. In IA-mode, this instruction is omitted.
   Run $\mathsf{Verify}(X; m, (r, s))$ to receive β. If $\beta = 0$, then output $\perp$ and halt.

– **Output:** (r, s)

6.3 Proof of Security

Ideal signing functionality. In Functionality 6, we define an ideal functionality for distributed ECDSA signing. Note that it receives four commands: KeyGen, preSign, Randomize and Sign. In the preSign step, a new nonce $R' = k' \cdot E$ is chosen and handed to the ideal-world adversary. However, once the message is known, the nonce is randomized using the Randomize command, which sets $R = \rho \cdot R' + \mu \cdot E$, where ρ is chosen by the adversary and μ is chosen randomly by the functionality. Finally, upon receiving the Sign command, the functionality computes the signature and hands it to the adversary. We note that the functionality allows the simulator to return a different valid signature. This captures also the version of ECDSA which does not have the strongly-unforgeable property. Clearly, when the strongly-unforgeable version of ECDSA is used, the adversary cannot produce a different signature for m. As noted above, ECDSA with presigning and such a randomization have been shown [1] to incur only a small constant loss.

Security. In the following theorem, we state the security of our protocol. Due to space constraints, the proof is in the full version of our paper.

Theorem 7. *Let $0 \leq t \leq n-1$ be a security threshold. Then, assuming $\mathcal{H}_\mu$ and $\mathcal{H}_\rho$ are modeled as a random oracle, Protocol 2+3 (distributed key generation) and Protocol 5 (distributed signing) securely compute $\mathcal{F}_{\mathsf{ECDSA}}$ with selective abort in the presence of malicious adversaries controlling up to t malicious parties.*

6.4 Extensions

Achieving Identifiable Abort We can easily transform Protocol 2 (setup) and Protocol 5 (signing) to achieve the stronger property of identifiable abort

Functionality 6 ($\mathcal{F}_{\mathsf{ECDSA}}$ - Distributed ECDSA)

- **Parties:** $P_1, \ldots, P_N$ and the ideal-world adversary $\mathcal{S}$.
- **Parameters:** $0 \leq t < N \in \mathbb{N}$ and $\kappa \in \mathbb{N}$.
- **Operation**]
  - Upon receiving $(\mathsf{sid}, \mathsf{KeyGen})$ from all n parties:
    1. Let $(x, X) \leftarrow \mathsf{KeyGen}(1^\kappa)$.
    2. Store (sid, x).
    3. Send (sid, X) to all parties.
    4. Ignore future calls withthe same sid.
  - Upon receiving $(\mathsf{sid}, \mathsf{preSign})$ from $t+1$ parties:
    1. Retrieve (sid, x).
    2. Choose a random $k' \in \mathbb{F}_q$ and set $R' = k' \cdot E$.
    3. Send (sid, R') to $\mathcal{S}$ and store (sid, x, k').
  - Upon receiving $(\mathsf{sid}, \mathsf{Randomize}, \rho, m)$ from $t+1$ parties:
    1. Retrieve (sid, x, k').
    2. Choose $\mu \in \mathbb{F}_q$ and compute $k = \rho \cdot k' + \mu$
    3. Set $R = k \cdot E$. Let r be the x-coordinate of R mod q.
    4. Send (sid, R, μ) to $\mathcal{S}$ and store $(\mathsf{sid}, x, r, k, m)$.
    5. Ignore any future call with same sid.
  - Upon receiving $(\mathsf{sid}, \mathsf{Sign}, m, r)$ from $t+1$ parties:
    1. Retrieve $(\mathsf{sid}, x, r, k, m)$.
    2. Compute $s = k^{-1} \cdot (\mathcal{H}(m) + r \cdot x)$
    3. Send $(\mathsf{sid}, (r, s))$ to $\mathcal{S}$.
    4. Wait to receive back (r, s') from $\mathcal{S}$. If $\mathsf{Verify}(X; m, (r, s')) = 1$ output (r, s') to all parties. Otherwise, output abort.

(IA). This is done when the instructions for IA-Mode (in blue) are executed and all messages are sent over a broadcast channel. In this case, as all messages are public and accompanied by zero-knowledge proofs, anyone observing the transcript is able to identify misbehaving participants simply by verifying the proofs.

Achieving identifiable abort for Protocol 3 (key generation) is not immediate however as it requires privately sending Shamir secret shares to the other participants. To overcome this, it is possible to use a publicly-verifiable VSS instead of the described VSS, which we leave the description of to existing works on this topic (as we do not impose any additional or special requirements and are designed to be interoperable with any Shamir-secret-shared signing key).

Achieving Robustness even in the Asynchronous Setting ROAST [25] is an efficient transformation which lifts certain threshold signing protocols to work over an asynchronous network and guarantee delivery of the output (a valid signature). The underlying protocol must meet three requirements, which our protocol satisfies: (i) The protocol is semi-interactive: two rounds, where the first round does not require the message being signed nor the signing set as an input. Our first round is key-, signing-set-, and message- independent. (ii) Support for

	Variable-Time	Constant-Time	Bytes
Presign	103ms	3,044ms	1,350
Sign	303ms	9,025ms	2,505
Sign without Identifiable Aborts	54ms	1,492ms	450

Table 2. The median time to participate in each round of our protocol for a transcript where messages have already been verified and aggregated (which does not require the witness) and the median size of each participant's message in bytes. These values are constant to the security parameter and independent of the set size.

	2-of-*	34-of-*	67-of-*
With Identifiable Aborts (Variable-Time)	366ms	1,608ms	2,599ms
Without Identifiable Aborts (Variable-Time)	195ms	633ms	1,085ms
With Identifiable Aborts (Constant-Time)	12,129ms	13,271ms	14,449ms
Without Identifiable Aborts (Constant-Time)	4,574ms	5,012ms	5,464ms

Table 3. The benchmarks of the signing protocol in its entirety, including the time to participate in each round and the time to verify and aggregate messages. The columns represent various set sizes with the median time taken per-participant. The constant-time variants are only constant in time with regards to the witness.

identifiable aborts: a party with the transcript can identify misbehaving parties. Our protocol offers this so long as the protocol is ran in IA-mode. (iii) The signing protocol is unforgeable even during concurrent executions, which our simulation-based proofs and method of presigning ensure. Hence, we can apply the ROAST transformation to Trout++ and obtain a robust protocol, even over an asynchronous network.

7 Implementation

Trout [13] was published with a constant-time library for class group cryptography in order to defend against side-channel attacks. We have significantly optimized this library and written a implementation of Trout++ from scratch[6].

Between the the increased efficiency of Trout++ and further work on the implementation, we have continued to close the gap between the hardened constant-time implementation and the protocol itself, making it more and more considerable for deployment.

Practical Optimizations. Like Trout, the implemented zero-knowledge proofs are derived from the methodology of [12]. We introduce a practical optimization for the elements in the response to the challenge denoted D. The CLT encryption scheme over class groups [10] requires a fundamental discriminant Δ_k and a discriminant Δ_q. Between these two class groups is a surjection π and injection ψ such that $q \cdot \psi(\pi(D)) = q \cdot D$. Instead of responding to the challenge with the element D of the class group with discriminant Δ_q, we respond with the element $\pi(D)$ of the class group with discriminant Δ_k, which has an encoding approximately 25% smaller and is allowed as the verification equation scales the element by q. This also restricts from having q valid responses to the challenge

[6] The code is available here: https://github.com/kayabaNerve/trout

to just one (under the computational assumptions already present), ensuring the response is canonical and without malleability. We are also able to take advantage of these maps with our optimized commitments as they lack any terms from $\hat{H}$.

Like Trout's, our implementation compresses binary quadratic forms as detailed in [14]. The compression algorithm requires, for numbers $b,\ a',\ g,\ f$ where $g \leq f,\ b \leq g \cdot a' \leq \mathsf{lcm}(f,\ a')$, sending $b \mod f,\ b \mod a'$ before using the Chinese Remainder Theorem to reconstruct b (with the exceptional case $b = a$ handled specially). Unfortunately, the algorithm only has a statistical bound for its termination due to having to find a suitable value for f which satisfies the requirements. We introduce a slight variant of the scheme where we instead send $\lfloor b/a' \rfloor,\ b \mod a'$, effectively representing b as its Euclidean division by a'. This avoids the f variable entirely, achieving a perfect (not statistical) bound on termination and the size of the resulting encoding, not to mention a simpler scheme overall.

Experimental Results. In Table 2 and Table 3, we present the performance of our implementation. All of the benchmarks were obtained on a Mac mini with an Apple M2 Pro CPU, each participant only using a single-thread to participate in the signing protocol. All tests were ran with a 128-bit computational security parameter and a 1827-bit fundamental discriminant. When sampling from distributions $\mathcal{D}_q,\ \mathcal{D}$, and when performing batch verification, a 128-bit statistical security parameter was used. For the variable-time invocations, [4] was used for the implementation of class group cryptography.

Comparison to Trout. The resulting implementation is almost twice as fast as Trout's implementation for participants with variable-time provers and approximately *ten times as fast* for participants with constant-time provers. Additionally, the median upload per-participant in the bulletin board model was reduced from 6,345 bytes to 3,855 bytes with identifiable aborts and from 3,902 bytes to 1,800 bytes without identifiable aborts. Some of this benefit is from the Trout++ protocol while some is the result of the improvements to the implementation. For a more direct comparison of the protocols themselves, we recommend comparing the variable-time column from Table 2 to Trout's published performance numbers for proving the round one and round two proofs in variable time. This eliminates any differences in the constant-time implementation by excluding it entirely, allowing focusing on the average case of the protocols themselves instead.

8 Key Derivation

Performing distributed key generation (DKG) or similar setup protocols is often significantly more expensive than executing the signing protocol itself. While this is reasonable—since setup is done once whereas signing may occur many times—there are situations where an entity wants or needs multiple signing keys. For example, distinct signing keys can be used to embed context into the signature (when the signed messages cannot effectively encode the context), or in privacy-preserving settings where multiple identities (public keys) are desirable.

Both motivations are illustrated by BIP 44[7], a Bitcoin standard in which public keys encode wallet structure, distinguishing accounts and whether funds originate from internal "change" outputs or external sources. Addresses within an account are further indexed, enabling many addresses per account. In practice, this leads to the recommendation of using each address only once, generating a fresh address for every interaction to improve privacy by reducing linkability.

When the user is a group performing DKG and signing protocols, the desire for multiple public keys persists, although traditional approaches to derive keys generally do not directly apply. In centralized settings, keys are typically derived from a single root private key, requiring only a single setup. In contrast, distributed settings generally rely on secret-shared keys, practically limiting derivations to those supported homomorphically.

Groth and Shoup [20] introduced affine ("homogenous") key derivation, where signing is performed with respect to keys of the form $X + d \cdot X'$ for $X,\ X' \in \hat{E}$ and a derivation d. They show that allowing such derivations gives an adversary only negligible additional advantage compared to standard ECDSA. Next, we show how it can be composed with Trout++.

Affine Derivation. The Trout++ Setup may be performed twice, once for $\mathsf{ppEC},\ \mathsf{ppCL},\ X',\ \{X'_i\}_{i=1}^N$ with private input x'_i and once for $\mathsf{ppEC},\ \mathsf{ppCL},\ X'',\ \{X''_i\}_{i=1}^N$ with private input x''_i. As $\mathsf{ppEC},\ \mathsf{ppCL}$ are consistent across inputs, they will be consistent across the outputs, with $X', X'', \{X'_i\}_{i=1}^N,\ \{X''_i\}_{i=1}^N \in \hat{E}$ and $\{C'_i\}_{i=1}^N, \{C''_i\}_{i=1}^N \in \hat{G}$. As the derivation process is linear, it is possible to obtain an output in the same class as the output from a single invocation of the Trout++ Setup, by computing for a derivation d: $X = X' + d \cdot X'',\ \{X_i\}_{i=1}^N = X'_i + d \cdot X''_i,\ \{C_i\}_{i=1}^N = C'_i + d \cdot C''_i,\ \delta_i = \delta'_i + d \cdot \delta''_i,\ x_i = x'_i + d \cdot x''_i$.

References

1. Adjedj, M., Blokh, C., Couteau, G., Galansky, A., Joux, A., Makriyannis, N.: Two-round 2pc ECDSA at the cost of 1 OLE: applications to embedded cryptocurrency wallets. In: Public-Key Cryptography - PKC. pp. 355–386 (2026)
2. Boneh, D., Gennaro, R., Goldfeder, S.: Using level-1 homomorphic encryption to improve threshold DSA signatures for bitcoin wallet security. In: LATINCRYPT. pp. 352–377 (2017)
3. Boneh, D., Haitner, I., Lindell, Y., Segev, G.: Exponent-vrfs and their applications. In: Advances in Cryptology - EUROCRYPT. pp. 195–224 (2025)
4. Bouvier, C., Castagnos, G., Imbert, L., Laguillaumie, F.: I want to ride my bicycl: Bicycl implements cryptography in class groups. IACR Cryptol. ePrint Arch., paper 2022/1466 (2022)
5. Boyle, E., Devadas, L., Servan-Schreiber, S.: Non-interactive distributed point functions. In: Public-Key Cryptography - PKC. pp. 3–35 (2025)
6. Bünz, B., Choi, K., Komlo, C.: Golden: Lightweight non-interactive distributed key generation. IACR Cryptol. ePrint Arch. (2025), `https://eprint.iacr.org/2025/1924`

[7] https://github.com/bitcoin/bips/blob/master/bip-0044.mediawiki

7. Canetti, R.: Security and composition of multiparty cryptographic protocols. J. Cryptol. **13**(1), 143–202 (2000)
8. Cascudo, I., David, B.: Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In: Advances in Cryptology - EUROCRYPT. pp. 216–248 (2024)
9. Castagnos, G., Catalano, D., Laguillaumie, F., Savasta, F., Tucker, I.: Two-party ECDSA from hash proof systems and efficient instantiations. In: Advances in Cryptology - CRYPTO. pp. 191–221 (2019)
10. Castagnos, G., Laguillaumie, F., Tucker, I.: Practical fully secure unrestricted inner product functional encryption modulo p. In: ASIACRYPT. pp. 733–764 (2018)
11. Cohen, H.: A Course in Computational Algebraic Number Theory. Springer Berlin, Heidelberg (1996)
12. Cui, H., Chan, K.Y., Yuen, T.H., Kang, X., Chu, C.: Bandwidth-efficient zero-knowledge proofs for threshold ECDSA. Comput. J. pp. 1265–1278 (2024)
13. Dahari-Garbian, H., Nof, A., Parker, L.: Trout: Two-round threshold ECDSA from class groups. In: ACM CCS. pp. 380–393 (2025)
14. Dobson, S., Galbraith, S.D., , Smith, B.: Trustless unknown-order groups. IACR Cryptol. ePrint Arch., paper 2020/196 (2020)
15. Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Threshold ECDSA from ECDSA assumptions: The multiparty case. In: IEEE Symposium on Security and Privacy, SP 2019. pp. 1051–1066 (2019)
16. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: Annual Symposium on Foundations of Computer Science. pp. 427–437 (1987)
17. Gennaro, R., Goldfeder, S.: Fast multiparty threshold ECDSA with fast trustless setup. In: ACM CCS. pp. 1179–1194 (2018)
18. Gennaro, R., Goldfeder, S., Narayanan, A.: Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In: Applied Cryptography and Network Security, ACNS. pp. 156–174 (2016)
19. Gouvêa, C.P.L., Komlo, C.: Re-randomized FROST. IACR Cryptol. ePrint Arch. (2024), `https://eprint.iacr.org/2024/436`
20. Groth, J., Shoup, V.: On the security of ECDSA with additive key derivation and presignatures. In: Advances in Cryptology - EUROCRYPT. pp. 365–396 (2022)
21. Komlo, C., Goldberg, I.: FROST: flexible round-optimized schnorr threshold signatures. In: Selected Areas in Cryptography - SAC. pp. 34–65 (2020)
22. Lindell, Y., Nof, A.: Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In: ACM CCS. pp. 1837–1854 (2018)
23. Lyu, Y., Li, Z., Zhou, H., Deng, X.: Threshold ECDSA in two rounds. In: ACM CCS. pp. 2937–2950 (2025)
24. Parker, L.: Robust Threshold eVRF DKG. Unpublished (2024), `https://github.com/serai-dex/serai/tree/next/audits/crypto/dkg/evrf`
25. Ruffing, T., Ronge, V., Jin, E., Schneider-Bensch, J., Schröder, D.: ROAST: robust asynchronous schnorr threshold signatures. In: ACM CCS. pp. 2551–2564 (2022)
26. Shamir, A.: How to share a secret. Commun. ACM **22**(11), 612–613 (1979)
27. Yuen, T.H., Cui, H., Xie, X.: Compact zero-knowledge proofs for threshold ECDSA with trustless setup. In: Public-Key Cryptography - PKC. pp. 481–511 (2021)